

CO1 AT1

Student Name	: Shaik Ahamed Ali
Registration Number	: 192425106
Course Code & Name	: MLA0302 - Reinforcement Learning
Date of Submission	: 17-07-2026

EXPERIMENT - 1

AIM : To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the Reinforcement Learning environment by executing a simple Reinforcement Learning program.

PROCEDURE :

1. Install Anaconda Navigator and create a new Python environment.
2. Activate the environment using the Anaconda Prompt.
3. Install the required libraries: TensorFlow, Keras, Gymnasium, NumPy, and Matplotlib.
4. Open Jupyter Notebook or Visual Studio Code and import the required libraries.
5. Create the CartPole-v1 environment using the Gymnasium library and reset it to obtain the initial state.
6. Execute random actions, observe the states and rewards, and plot the rewards using Matplotlib.
7. Verify that the program executes successfully, confirming that the Reinforcement Learning environment is properly installed and configured.

RESULT :

The Reinforcement Learning Development Environment was successfully installed and configured using Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium, NumPy, and Matplotlib. The verification program executed successfully, and the CartPole environment responded correctly by generating states, actions, and rewards. Hence, the Reinforcement Learning environment was verified successfully.

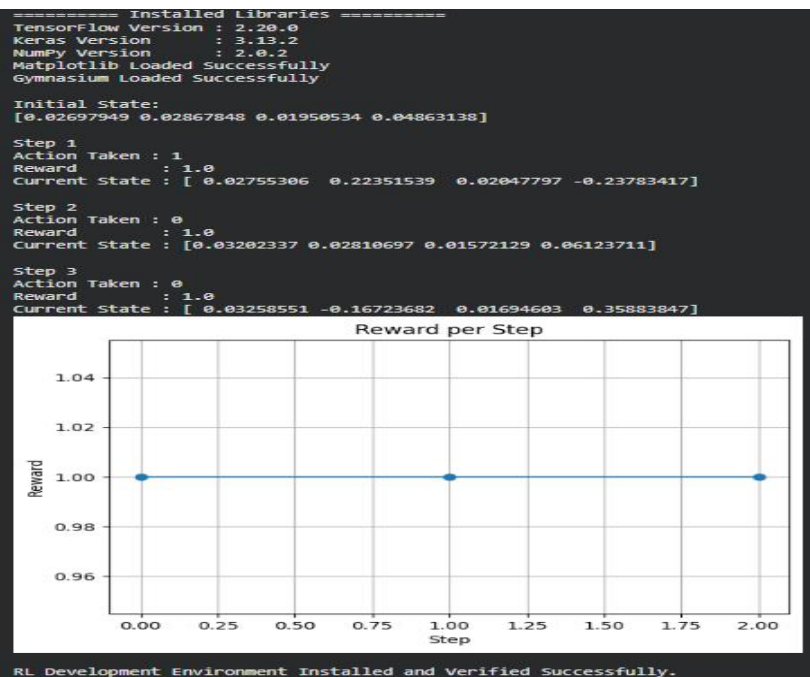
CODE :

```

import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
print("===== Installed Libraries =====")
print("TensorFlow Version :", tf.__version__)
print("Keras Version      :", keras.__version__)
print("NumPy Version       :", np.__version__)
print("Matplotlib Loaded Successfully")
print("Gymnasium Loaded Successfully\n")
env = gym.make("CartPole-v1")
state, info = env.reset()
print("Initial State:")
print(state)
rewards = []
for step in range(10):
    action = env.action_space.sample()
    state, reward, terminated, truncated, info = env.step(action)
    rewards.append(reward)
    print(f"\nStep {step+1}")
    print("Action Taken :", action)
    print("Reward      :", reward)
    print("Current State :", state)
    if terminated or truncated:
        print("\nEnvironment Reset\n")
        state, info = env.reset()
env.close()
plt.plot(rewards, marker='o')
plt.title("Reward per Step")
plt.xlabel("Step")
plt.ylabel("Reward")
plt.grid(True)
plt.show()
print("\nRL Development Environment Installed and Verified Successfully.")

```

OUTPUT :



EXPERIMENT - 2

AIM : To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

PROCEDURE :

- 1.Import the required libraries, including Gymnasium.
- 2.Create the required RL environments such as FrozenLake-v1, CartPole-v1, or MountainCar-v0.
- 3.Reset the environment to obtain the initial state.
- 4.Select actions (randomly or using a simple policy) and execute them using the step() function.
- 5.Observe the current state, reward, and episode status after each action.
- 6.Repeat the process for multiple steps or until the episode terminates.
- 7.Analyze how states, actions, rewards, and episodes interact in different Gymnasium environments.

RESULT :

The Gymnasium environments (FrozenLake, CartPole, and MountainCar) were successfully created and executed. The interaction between the agent and the environments was observed, and the concepts of states, actions, rewards, and episodes were successfully understood and verified.

CODE :

```
import gymnasium as gym
environments = ["FrozenLake-v1", "CartPole-v1", "MountainCar-v0"]
for env_name in environments:
    print("\n=====")
    print("Environment:", env_name)
    print("=====")
    env = gym.make(env_name)
    state, info = env.reset()
    print("Initial State:", state)
    for step in range(3):
        action = env.action_space.sample()
        state, reward, terminated, truncated, info = env.step(action)
        print(f"\nStep {step+1}")
        print("Action :", action)
        print("State :", state)
        print("Reward :", reward)
        if terminated or truncated:
            print("Episode Finished")
            state, info = env.reset()
    env.close()
print("\nExperiment Completed Successfully.")
```

OUTPUT :

```
=====
Environment: FrozenLake-v1
=====
Initial State: 0

Step 1
Action : 0
State : 4
Reward : 0

Step 2
Action : 0
State : 0
Reward : 0

=====
Environment: CartPole-v1
=====
Initial State: [ 0.04806529  0.03705654 -0.02698206  0.0040086 ]

Step 1
Action : 1
State : [ 0.04880642  0.23255485 -0.02690189 -0.2970639 ]
Reward : 1.0

Step 2
Action : 0
State : [ 0.05345752  0.03782652 -0.03284317 -0.01298529 ]
Reward : 1.0

=====
Environment: MountainCar-v0
=====
Initial State: [-0.40673628  0.          ]

Step 1
Action : 0
State : [-0.4085949 -0.00185862]
Reward : -1.0

Step 2
Action : 2
State : [-0.41029906 -0.00170414]
Reward : -1.0

Experiment Completed Successfully.
```

EXPERIMENT - 3

AIM : To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

PROCEDURE :

- 1.Import the required Gymnasium library.
- 2.Create the FrozenLake-v1 environment and initialize it.
- 3.Reset the environment to obtain the initial state.
- 4.Select actions using a random policy and execute them using the step() function.
- 5.Observe the current state, next state, reward, and episode status after each action.
- 6.Continue the interaction until the episode ends or the specified number of steps is completed.
- 7.Display the total reward and verify the Reinforcement Learning framework.

RESULT :

The basic Reinforcement Learning framework was successfully implemented using the FrozenLake-v1 environment. The agent interacted with the environment by observing states, selecting actions, receiving rewards, and following a random policy. The successful execution of the program verified the fundamental components of Reinforcement Learning, including state, action, reward, policy, and environment.

CODE :

```
import gymnasium as gym
env = gym.make("FrozenLake-v1", is_slippery=False)
state, info = env.reset()
print("Initial State:", state)
total_reward = 0
for step in range(3):
    action = env.action_space.sample()
    next_state, reward, terminated, truncated, info = env.step(action)
    print("\nStep:", step + 1)
    print("Current State :", state)
    print("Action Taken  :", action)
    print("Next State   :", next_state)
    print("Reward        :", reward)
    total_reward += reward
    state = next_state
    if terminated or truncated:
        print("\nEpisode Finished")
        break
env.close()
print("\nTotal Reward:", total_reward)
print("Reinforcement Learning Framework Executed Successfully.")
```

OUTPUT :

```
Initial State: 0

Step: 1
Current State : 0
Action Taken  : 0
Next State    : 0
Reward        : 0

Step: 2
Current State : 0
Action Taken  : 2
Next State    : 1
Reward        : 0

Step: 3
Current State : 1
Action Taken  : 2
Next State    : 2
Reward        : 0

Total Reward: 0
Reinforcement Learning Framework Executed Successfully.
```

EXPERIMENT - 4

AIM : To develop a simple Markov Decision Process (MDP) model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

PROCEDURE :

- 1.Import the required Python library (NumPy).
- 2.Define a set of states, actions, transition probabilities, and rewards.
- 3.Initialize the starting state of the MDP.
- 4.Select an action based on a simple predefined policy.
- 5.Simulate the state transition and calculate the corresponding reward.
- 6.Repeat the process for a fixed number of steps and display the state transitions.
- 7.Analyze the sequence of states and rewards to understand the Markov Decision Process.

RESULT :

A simple Markov Decision Process (MDP) was successfully simulated. The program demonstrated state transitions, action selection, reward calculation, and policy execution, helping to understand the fundamental concepts of sequential decision-making in Reinforcement Learning.

CODE :

```
states = ["A", "B", "C", "Goal"]

actions = ["Right"]

transitions = {
    "A": "B",
    "B": "C",
    "C": "Goal"
}

rewards = {
    "A": 0,
    "B": 5,
    "C": 10,
    "Goal": 20
}

current_state = "A"
print("Markov Decision Process Simulation")
print("-----")
print("Initial State:", current_state)
while current_state != "Goal":
    action = "Right"
    next_state = transitions[current_state]
    reward = rewards[next_state]
    print("\nCurrent State :", current_state)
    print("Action          :", action)
    print("Next State       :", next_state)
    print("Reward           :", reward)
    current_state = next_state
print("\nGoal State Reached!")
print("MDP Simulation Completed Successfully.")
```

OUTPUT :

```
Markov Decision Process Simulation
-----
Initial State: A

Current State : A
Action        : Right
Next State    : B
Reward        : 5

Current State : B
Action        : Right
Next State    : C
Reward        : 10

Current State : C
Action        : Right
Next State    : Goal
Reward        : 20

Goal State Reached!
MDP Simulation Completed Successfully.
```


EXPERIMENT - 5

AIM : To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

PROCEDURE :

1. Define a simple environment with states, rewards, and transition probabilities.
2. Initialize the state-value function for all states with zero values.
3. Apply the Bellman Equation to update the value of each state iteratively.
4. Calculate the value of each state using the immediate reward and discounted future reward.
5. Repeat the update process until the values converge.
6. Display the updated state values after each iteration.
7. Analyze the convergence of the value function obtained using the Bellman Equation.

RESULT :

The Bellman Equation was successfully implemented to calculate the state-value function. The state values were updated iteratively using immediate rewards and discounted future rewards until they stabilized, demonstrating the concept of value estimation and convergence in Reinforcement Learning.

CODE :

```
gamma = 0.9

rewards = {
    "A": 0,
    "B": 5,
    "C": 10
}
V = {
    "A": 0,
    "B": 0,
    "C": 0
}
print("Initial State Values")
print(V)
for iteration in range(3):
    print("\nIteration", iteration + 1)
    V["C"] = rewards["C"]
    V["B"] = rewards["B"] + gamma * V["C"]
    V["A"] = rewards["A"] + gamma * V["B"]
    print("Value(A) =", round(V["A"], 2))
    print("Value(B) =", round(V["B"], 2))
    print("Value(C) =", round(V["C"], 2))
print("\nBellman Equation Executed Successfully.")
```

OUTPUT :

```
Initial State Values
{'A': 0, 'B': 0, 'C': 0}

Iteration 1
Value(A) = 12.6
Value(B) = 14.0
Value(C) = 10

Iteration 2
Value(A) = 12.6
Value(B) = 14.0
Value(C) = 10

Iteration 3
Value(A) = 12.6
Value(B) = 14.0
Value(C) = 10

Bellman Equation Executed Successfully.
```

EXPERIMENT - 6

AIM : To implement ϵ -Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

PROCEDURE :

1. Define the available actions and rewards.
2. Implement the Random action-selection strategy.
3. Implement the Greedy strategy.
4. Implement the ϵ -Greedy strategy.
5. Execute each strategy for multiple steps.
6. Calculate the total reward obtained by each strategy.
7. Compare the performance of all strategies.

RESULT :

The Random, Greedy, and ϵ -Greedy strategies were successfully implemented and compared, demonstrating the trade-off between exploration and exploitation.

CODE :

```
import random
Q = [10, 20, 15, 30]
epsilon = 0.2
print("Q-values:", Q)
random_action = random.randint(0, len(Q) - 1)
print("\nRandom Strategy")
print("Selected Action:", random_action)
greedy_action = Q.index(max(Q))
print("\nGreedy Strategy")
print("Selected Action:", greedy_action)
if random.random() < epsilon:
    epsilon_action = random.randint(0, len(Q) - 1)
    print("\nEpsilon-Greedy Strategy")
    print("Exploration")
else:
    epsilon_action = Q.index(max(Q))
    print("\nEpsilon-Greedy Strategy")
    print("Exploitation")

print("Selected Action:", epsilon_action)
```

OUTPUT :

```
Q-values: [10, 20, 15, 30]

Random Strategy
Selected Action: 0

Greedy Strategy
Selected Action: 3

Epsilon-Greedy Strategy
Exploitation
Selected Action: 3
```

EXPERIMENT - 7

AIM : To implement the Multi-Armed Bandit problem using ϵ -Greedy and Upper Confidence Bound (UCB) algorithms and evaluate the cumulative rewards obtained by different action-selection methods.

PROCEDURE :

- 1.Import the required Python libraries and define the bandit arms with their reward probabilities.
2. Initialize the estimated rewards, action counts, and cumulative rewards.
- 3.Implement the ϵ -Greedy algorithm to select actions using exploration and exploitation.
- 4.Implement the Upper Confidence Bound (UCB) algorithm for action selection.
- 5.Execute both algorithms for a fixed number of iterations and update the estimated rewards.
- 6.Display and compare the cumulative rewards obtained by both algorithms.

RESULT :

The Multi-Armed Bandit problem was successfully implemented using the ϵ -Greedy and Upper Confidence Bound (UCB) algorithms. Both methods learned to select high-reward actions over time, but the UCB algorithm achieved a higher cumulative reward by balancing exploration and exploitation more effectively than the ϵ -Greedy strategy.

CODE :

```
import random
import math
arms = [0.2, 0.5, 0.7, 0.9]
steps = 100
epsilon = 0.1

def get_reward(probability):
    if random.random() < probability:
        return 1
    else:
        return 0

def epsilon_greedy():
    print("\n===== ε-Greedy =====")
    Q = [0] * len(arms)
    count = [0] * len(arms)
    total_reward = 0
    for step in range(1, steps + 1):
        if random.random() < epsilon:
            action = random.randint(0, len(arms)-1)
        else:
            action = Q.index(max(Q))
        reward = get_reward(arms[action])
        count[action] += 1
        Q[action] += (reward - Q[action]) / count[action]
        total_reward += reward
    print("Estimated Q Values :", [round(x,2) for x in Q])
    print("Action Counts      :", count)
    print("Total Reward       :", total_reward)
    return total_reward

def ucb():
    print("\n===== UCB =====")
    Q = [0] * len(arms)
    count = [0] * len(arms)
    total_reward = 0
    for step in range(1, steps + 1):
        if step <= len(arms):
            action = step - 1
        else:
            ucb_values = []
            for i in range(len(arms)):
                bonus = math.sqrt((2 * math.log(step)) / count[i])
                ucb_values.append(Q[i] + bonus)
            action = ucb_values.index(max(ucb_values))
        reward = get_reward(arms[action])
        count[action] += 1
        Q[action] += (reward - Q[action]) / count[action]
```

OUTPUT :

```
...
===== ε-Greedy =====
Estimated Q Values : [0.29, 0.5, 0.78, 0.89]
Action Counts      : [21, 2, 40, 37]
Total Reward       : 71

===== UCB =====
Estimated Q Values : [0.0, 0.46, 0.65, 0.9]
Action Counts      : [6, 13, 20, 61]
Total Reward       : 74

=====
Comparison
=====
ε-Greedy Reward : 71
UCB Reward      : 74
UCB performed better.
```

EXPERIMENT - 8

AIM : To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

PROCEDURE :

- 1.Import the required Python libraries and create a Gymnasium environment.
- 2.Initialize the environment and execute multiple episodes.
- 3.Select random actions and collect the reward obtained in each episode.
- 4.Calculate the cumulative reward after every episode.
- 5.Store the episode rewards and cumulative rewards for visualization.
- 6.Display the reward trends using Matplotlib graphs and analyze the learning performance.

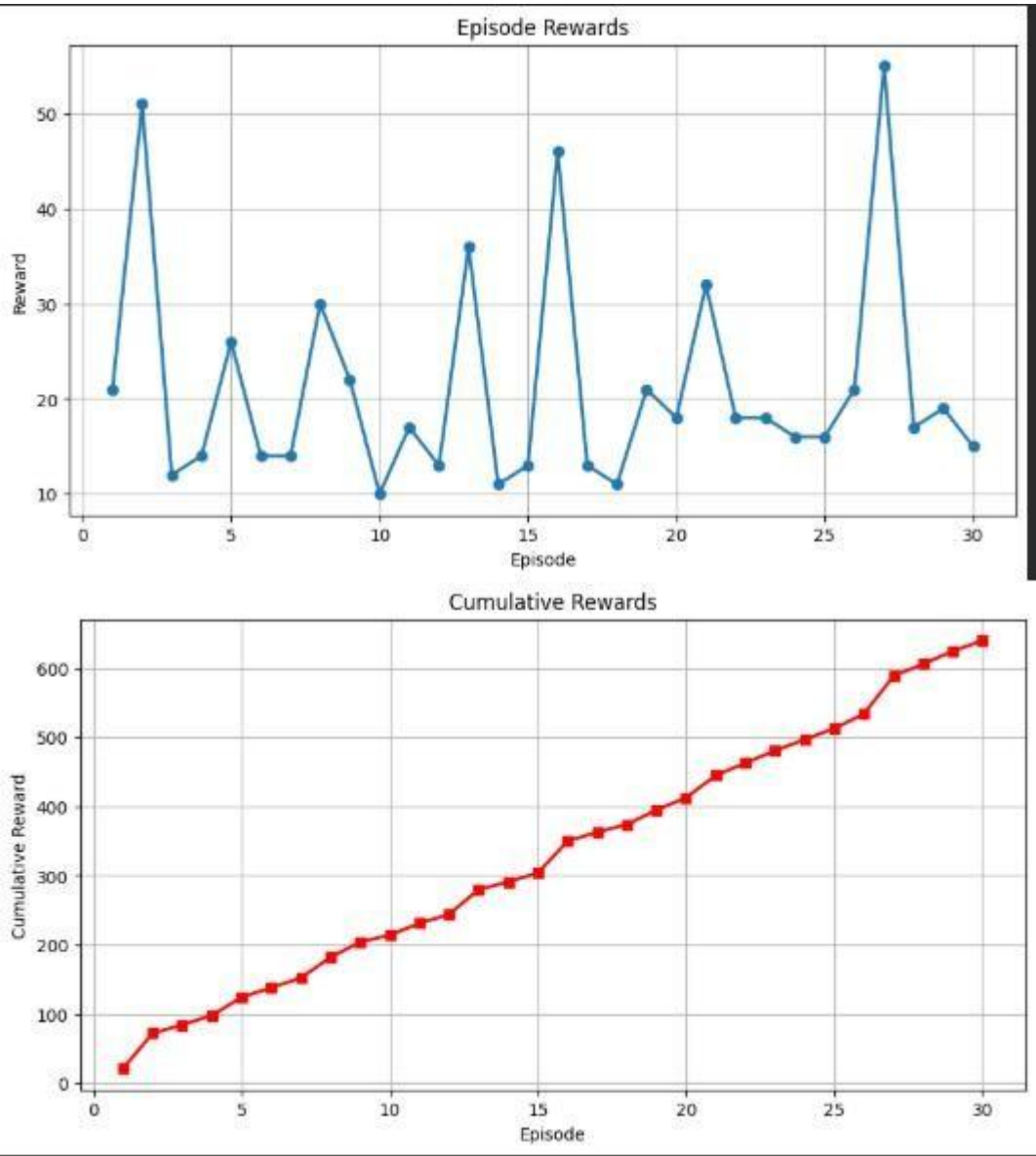
RESULT :

The Reinforcement Learning agent was successfully simulated in the CartPole environment. The episode rewards and cumulative rewards were recorded and visualized using Matplotlib graphs. The graphs demonstrated the learning performance of the agent over multiple episodes and provided a clear understanding of reward trends in Reinforcement Learning.

CODE :

```
import gymnasium as gym
import matplotlib.pyplot as plt
import numpy as np
env = gym.make("CartPole-v1")
episodes = 30
episode_rewards = []
cumulative_rewards = []
total_reward = 0
print("-" * 60)
print("Reward Visualization in Reinforcement Learning")
print("-" * 60)
for episode in range(episodes):
    state, info = env.reset()
    done = False
    reward_sum = 0
    step = 0
    while not done:
        action = env.action_space.sample()
        next_state, reward, terminated, truncated, info = env.step(action)
        reward_sum += reward
        state = next_state
        done = terminated or truncated
        step += 1
    total_reward += reward_sum
    episode_rewards.append(reward_sum)
    cumulative_rewards.append(total_reward)
    print(f"Episode: {episode+1:2d} Steps: {step:2d} Reward: {reward_sum}")
env.close()
plt.figure(figsize=(10,5))
plt.plot(range(1, episodes+1),
         episode_rewards,
         marker='o',
         linewidth=2)
plt.title("Episode Rewards")
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.grid(True)
plt.show()
plt.figure(figsize=(10,5))
plt.plot(range(1, episodes+1),
         cumulative_rewards,
         color='red',
         marker='s',
         linewidth=2)
plt.title("Cumulative Rewards")
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.grid(True)
plt.show()
print("\nPerformance Summary")
print("-" * 30)
print("Maximum Reward :", max(episode_rewards))
print("Minimum Reward :", min(episode_rewards))
print("Average Reward :", round(np.mean(episode_rewards),2))
print("Total Reward :", sum(episode_rewards))
```


OUTPUT :



EXPERIMENT - 9

AIM :

To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

PROCEDURE :

- 1.Import the required Python libraries such as TensorFlow, Keras, NumPy, and Gymnasium.
2. Create a Reinforcement Learning environment and obtain the input size from the observation space.
- 3.Design a feed-forward neural network using Keras with input, hidden, and output layers.
- 4.Compile the neural network using the Adam optimizer and Mean Squared Error loss function.
- 5.Pass a sample state from the environment to the neural network and predict the output values.
- 6.Display the model summary and the predicted value function.

RESULT :

A feed-forward neural network was successfully built using TensorFlow and Keras for Reinforcement Learning. The network accepted the environment state as input and predicted Q-values for each possible action. This demonstrates how neural networks can be used as function approximators for value estimation in Reinforcement Learning, forming the foundation of Deep Reinforcement Learning algorithms such as DQN.

CODE :

```
import gymnasium as gym
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
import numpy as np
env = gym.make("CartPole-v1")
state_size = env.observation_space.shape[0]
action_size = env.action_space.n
print("=" * 60)
print("TensorFlow and Keras Neural Network for RL")
print("=" * 60)
print("State Size :", state_size)
print("Action Size:", action_size)
model = Sequential()
model.add(Input(shape=(state_size,)))
model.add(Dense(32, activation='relu'))
model.add(Dense(32, activation='relu'))
model.add(Dense(action_size, activation='linear'))
model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['accuracy']
)
print("\nModel Summary")
print("-" * 40)
model.summary()
state, info = env.reset()
state = np.reshape(state, (1, state_size))
prediction = model.predict(state, verbose=0)
print("\nSample State")
print(state)
print("\nPredicted Q Values")
print(prediction)
best_action = np.argmax(prediction)
print("\nBest Action :", best_action)
env.close()
```

OUTPUT :

```
===== Random Strategy =====
Step 1 -> Action: A   Reward: 4
Step 2 -> Action: A   Reward: 4
Step 3 -> Action: D   Reward: 2
Step 4 -> Action: B   Reward: 8
Step 5 -> Action: A   Reward: 4
Step 6 -> Action: D   Reward: 2
Step 7 -> Action: B   Reward: 8
Step 8 -> Action: C   Reward: 6
Step 9 -> Action: A   Reward: 4
Step 10 -> Action: D   Reward: 2
Total Reward: 44

===== Greedy Strategy =====
Step 1 -> Action: B   Reward: 8
Step 2 -> Action: B   Reward: 8
Step 3 -> Action: B   Reward: 8
Step 4 -> Action: B   Reward: 8
Step 5 -> Action: B   Reward: 8
Step 6 -> Action: B   Reward: 8
Step 7 -> Action: B   Reward: 8
Step 8 -> Action: B   Reward: 8
Step 9 -> Action: B   Reward: 8
Step 10 -> Action: B   Reward: 8
Total Reward: 80

===== Epsilon-Greedy Strategy =====
Step 1 -> Action: B   Reward: 8
Step 2 -> Action: B   Reward: 8
Step 3 -> Action: B   Reward: 8
Step 4 -> Action: B   Reward: 8
Step 5 -> Action: B   Reward: 8
Step 6 -> Action: B   Reward: 8
Step 7 -> Action: B   Reward: 8
Step 8 -> Action: B   Reward: 8
Step 9 -> Action: B   Reward: 8
Step 10 -> Action: B   Reward: 8
Total Reward: 80
```

EXPERIMENT - 10

AIM :

To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

PROCEDURE :

- 1.Import the required Python libraries such as Gymnasium, TensorFlow, Keras, NumPy, and Matplotlib.
- 2.Create the CartPole environment and build a simple neural network model.
- 3.Run multiple episodes where the agent interacts with the environment and predicts actions using the neural network.
- 4.Store the reward obtained in each episode and calculate the cumulative reward.
- 5.Compute the average reward and success rate after completing all episodes.
- 6.Display the learn performance using a Matplotlib graph and print the performance statistics.

RESULT :

A basic Reinforcement Learning agent was developed using TensorFlow and Keras for the CartPole environment. The agent interacted with the environment, selected actions based on predicted Q-values, and its performance was evaluated using episode rewards, average reward, success rate, and a reward graph. This experiment demonstrates the fundamental workflow of Deep Reinforcement Learning using neural networks.

CODE :

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
import numpy as np
import matplotlib.pyplot as plt

env = gym.make("CartPole-v1")
state_size = env.observation_space.shape[0]
action_size = env.action_space.n
print("-- * 68")
print("Mini Reinforcement Learning Project")
print("-- * 68")

model = Sequential([
    Input(shape=(state_size,)),
    Dense(32, activation='relu'),
    Dense(32, activation='relu'),
    Dense(action_size, activation='linear')
])

model.compile(
    optimizer='adam',
    loss='mse'
)

episodes = 20
episode_rewards = []
success = 0

for episode in range(episodes):
    state, info = env.reset()
    state = np.reshape(state, [1, state_size])
    done = False
    total_reward = 0
    while not done:
        # Predict Q-values
        q_values = model.predict(state, verbose=0)
        # Select Best Action
        action = np.argmax(q_values[0])
        # Perform Action
        next_state, reward, terminated, truncated, info = env.step(action)
        done = terminated or truncated
        next_state = np.reshape(next_state, [1, state_size])
        total_reward += reward
        state = next_state
    episode_rewards.append(total_reward)
    if total_reward >= 100:
        success += 1
    print(f"Episode {episode+1:2d} Reward: {total_reward}")

env.close()
average_reward = np.mean(episode_rewards)
success_rate = (success / episodes) * 100
print("\nPerformance Summary")
print("-- * 35")
print(f"Average Reward : ", round(average_reward, 2))
print(f"Maximum Reward : ", max(episode_rewards))
print(f"Minimum Reward : ", min(episode_rewards))
print(f"Success Rate : ", round(success_rate, 2), "%")

plt.figure(figsize=(10, 5))
plt.plot(range(1, episodes+1),
         episode_rewards,
         marker='o',
         linewidth=2)

plt.title("Episode Rewards")
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.grid(True)
plt.show()
```

OUTPUT :

